Virtual Worlds

Programming Assignment #2 Report



학과/학부	소프트웨어학부
학번	2021203015
이름	강선후

2025. 5. 20

■ Fulfill table

Easy 1	First Person View Movement	O
Easy 2	Setting up Obstacles	O
Easy 3	Animal Spawn	O
Easy 4	Mini-map 1	O
Medium 1	Animal Hunger Bar	O
Medium 2	Mini-map 2	O
Hard	Feeding Animals by Hand	O
Expert 1	Move freely	O
Expert 2	Mini-map 3	X

■ For each requirement, explain how you fulfilled it

☐ Easy 1: First Person View Movement

```
void Move()
{
    float moveX = Input.GetAxis("Horizontal");
    float moveZ = Input.GetAxis("Vertical");
    float moveY = 0f;

    Vector3 move = transform.right * moveX + transform.forward * moveZ + transform.up * moveY;
    controller.Move(move * moveSpeed * Time.deltaTime);

    float horizontalSpeed = new Vector2(moveX, moveZ).magnitude;
    if (animator != null)
    {
        animator.SetFloat("Speed_f", horizontalSpeed);
    }
}
```

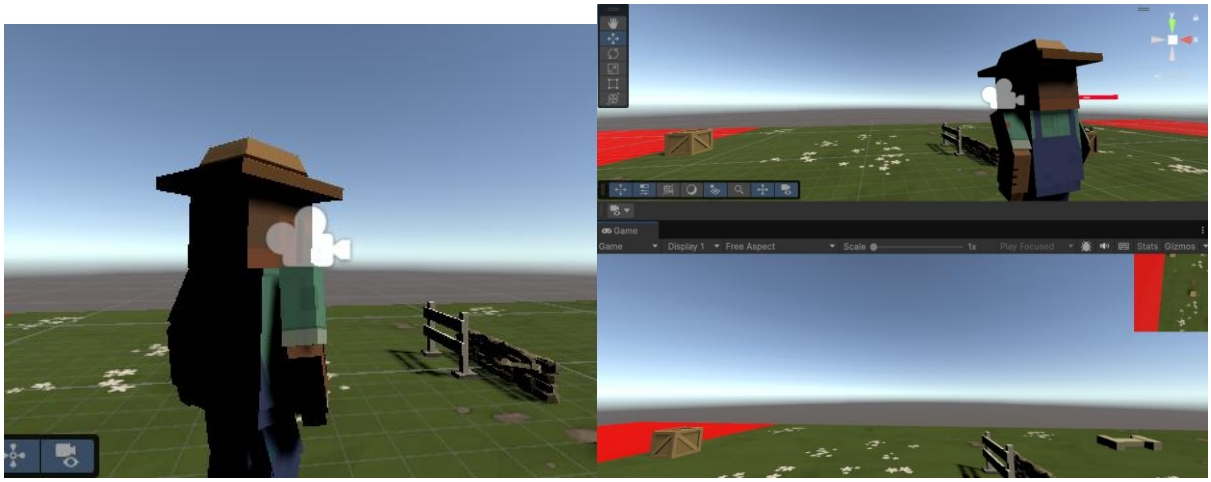
Player가 입력을 받아 맵 위를 돌아다닐 수 있게 구현하였고, 일정 속도 이상이 되면 walk 애니메이션이 적용되도록 하였습니다.

```
void LookAround()
{
    float mouseX = Input.GetAxis("Mouse X") * mouseSensitivity;
    float mouseY = Input.GetAxis("Mouse Y") * mouseSensitivity;

    verticalRotation -= mouseY;
    verticalRotation = Mathf.Clamp(verticalRotation, -verticalLookLimit, verticalLookLimit);

    cameraTransform.localRotation = Quaternion.Euler(verticalRotation, 0f, 0f);
    transform.Rotate(Vector3.up * mouseX);
}
```

Player에 Camera를 추가로 달아주고, 마우스를 이용하여 Player가 주변을 돌아볼 수 있도록 하였습니다.



☐ Easy 2: Setting up Obstacles

*사진은 각각 Easy2와 Easy3의 예시 사진입니다.





```
for (int i = 0; i < obstacleCount; i++)
{
    Vector3 randomPos = new Vector3(
        Random.Range(-spawnAreaSize.x / 2, spawnAreaSize.x / 2),
        0f,
        Random.Range(-spawnAreaSize.z / 2, spawnAreaSize.z / 2)
    );

    int randomIndex = Random.Range(0, obstaclePrefabs.Length);
    GameObject obstacle = Instantiate(obstaclePrefabs[randomIndex], randomPos, Quaternion.identity);
```

Play 시 GameObject[] obstaclePrefabs에 포함된 장애물 프리펩들이 무작위로 선택되고, 정해진 영역 내에 랜덤한 포지션에 생성될 수 있도록 하였습니다.

☐ Easy 3: Animal Spawn

Animal Spawn 역시 [Easy 2]와 유사하게 Play 시 5초 간격으로 GameObject[] animalPrefabs 에 포함된 동물 프리펩들이 정해진 영역 내에 랜덤한 포지션에 생성될 수 있도록 하였습니다.

```
for (int i = 0; i < 2; i++)
{
    GameObject prefab = animalPrefabs[Random.Range(0, animalPrefabs.Length)];
    Vector3 pos = new Vector3(Random.Range(-10f, 10f), 0f, Random.Range(-10f, 10f));
    GameObject animalObj = Instantiate(prefab, pos, Quaternion.identity);
```

☐ Easy 4: Mini-map 1

맵을 보여줄 minimapCamera를 설치하고, 미니맵을 위한 UI를 구현하였습니다. MiniMapController에서 플레이어 위치를 따라가며, Quaternion.Euler를 활용해 회전값을 동적으로 조정하였습니다.

```
if (rotateMiniMap) // Mini-map 1
{
    miniMapCamera.rotation = Quaternion.Euler(90, player.eulerAngles.y, 0);
}
```

플레이어는 minimap UI에서 보이지 않도록 하였으며, 동물들은 prefab 상단에 붉은색 실린더를 두어 minimap에서 붉은 원으로 표시될 수 있게 하였습니다. 이 때, 붉은 색 실린더는 레이어를 설정해주고 main camera의 culling mask에서 제외하여 Player의 시야에는 보이지 않게 하였습니다.



☐ Medium 1: Animal Hunger Bar

각 동물 위에 배치되는 Hunger Bar UI를 구현하였습니다.

동물 생성 시 바는 가득 차 있으며, 시간이 지남에 따라 감소하고, 0이 되면 동물과 UI가 모두 제거됩니다.

카메라를 항상 바라보도록 LateUpdate()에서 LookAt 로직을 구현하였고, Feed() 함수를 통해 바를 회복할 수 있도록 했습니다.

fillImage는 Image 컴포넌트의 fillAmount 속성을 활용해 현재 상태를 실시간으로 반영합니다.

```
currentHunger -= Time.deltaTime * 5f;

if (currentHunger <= 0f)
{
    Destroy(parentAnimal.gameObject);
    Destroy(gameObject);
}
UpdateBar();
```

```
Vector3 lookDirection = transform.position - cameraTransform.position;
transform.rotation = Quaternion.LookRotation(lookDirection, Vector3.up);
```

☐ Medium 2: Road Layout

고정 방향의 미니맵을 구현하였으며, R 키를 눌러 기존 회전형 미니맵과 토글할 수 있도록 했습니다. 회전형 모드에서는 플레이어 방향에 따라 카메라가 회전하고, 고정형 모드에서는 카메라는 항상 같은 방향을 유지합니다.



□ Hard: Feeding Animals by Hand

우클릭 시 음식 종류(Bone, Apple, Carrot)를 순차적으로 변경하며, 현재 선택된 음식은 우하단 UI에 표시됩니다. 좌클릭 시 해당 음식이 생성되어 전방으로 날아가고, 올바른 음식이 동물과 충돌하면 동물이 먹는 애니메이션을 재생하며 일정 시간 동안 이동을 멈춥니다. AnimalFeedable에서 OnTriggerEnter를 통해 음식 타입을 확인하고, Animal 스크립트를 활용해 정지 및 재이동을 제어하였습니다. FeederController에서 음식 프리팹을 동적으로 생성하고 Rigidbody의 AddForce로 투척하며, FoodProjectile 컴포넌트를 통해 음식 타입 정보를 전달합니다.



```
private void ThrowFood()
{
    GameObject foodObj = Instantiate(GetFoodPrefab(currentFood), foodSpawnPoint.position, Quaternion.identity);

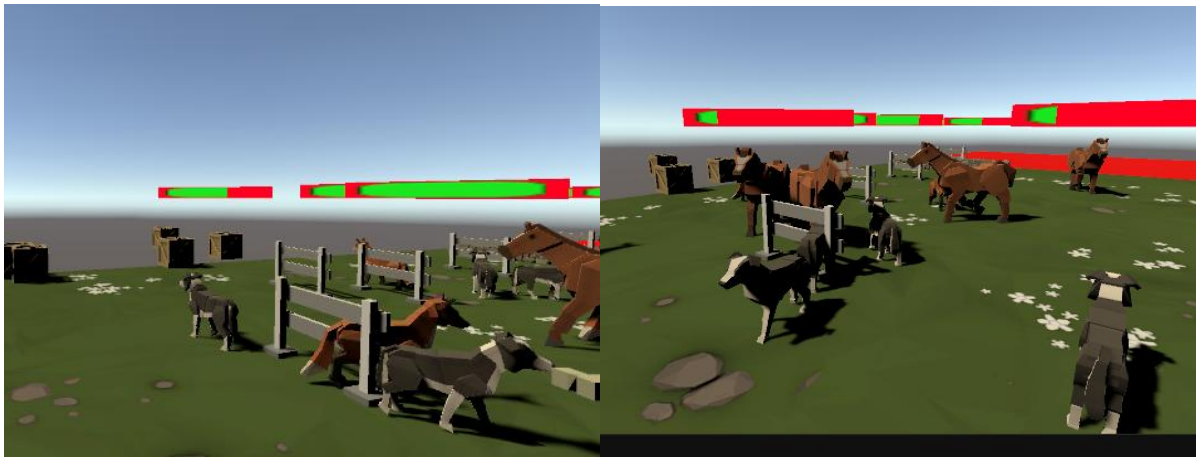
    Rigidbody rb = foodObj.GetComponent<Rigidbody>();
    if (rb == null)
        rb = foodObj.AddComponent<Rigidbody>();

    rb.useGravity = true;
    rb.collisionDetectionMode = CollisionDetectionMode.Continuous;
    rb.AddForce(foodSpawnPoint.forward * throwForce, ForceMode.VelocityChange);

    FoodProjectile projectile = foodObj.GetComponent<FoodProjectile>();
```

□ Expert: Move freely

Animal 스크립트에서 동물이 자유롭게 이동하도록 구현하였습니다. 일정 시간 간격으로 ChooseNewDirection() 함수를 통해 방향을 무작위로 갱신하며, Vector2.insideUnitCircle을 사용해 수평 방향으로 이동합니다. 이동 방향에 따라 Quaternion.LookRotation으로 자연스럽게 회전하도록 하였습니다. Update()에서 이동 벡터를 위치에 더하는 방식으로 움직임을 구현하였습니다.



```
void ChooseNewDirection()
{
    Vector2 random2D = Random.insideUnitCircle.normalized;
    moveDirection = new Vector3(random2D.x, 0, random2D.y);

    if (moveDirection != Vector3.zero)
    {
        transform.rotation = Quaternion.LookRotation(moveDirection);
    }
```

■ Conclude with some comments on your work

☐ Key challenges you have successfully tackled

동물이 올바른 먹이를 먹고 애니메이션을 재생하는 기능을 비롯하여, 미니맵 3를 제외한 대부분의 기능을 성공적으로 구현할 수 있었습니다. 특히 먹이를 던지고, 해당 먹이가 동물과 충돌했을 때 올바른 먹이인지 판단하여 먹는 동작과 이동 중단, 애니메이션 재생 등을 자연스럽게 구현하는 부분에서 많은 테스트와 디버깅이 필요했습니다.

☐ Limitations you hope to address in the future

현재 구현에서는 먹이를 던질 때 일정한 방향과 속도로만 발사되며, 동물의 위치나 방향을 고려한 보정 기능은 없습니다. 이로 인해 근접해 있어도 먹이가 동물의 충돌 범위에서 벗어나는 경우가 발생할 수 있습니다. 향후에는 동물의 위치나 방향을 반영하여 투척 방향을 자동 조정하거나, 충돌 판정 범위를 상황에 따라 유동적으로 조절하는 방식을 도입할 수 있으면 좋을 것 같습니다.

■ Add a link to a video that runs the program (e.g. YouTube)

https://youtu.be/fkWtsKwjKOE